

A Project Synopsis on
**Iot-Enabled Battery Health & Performance Monitoring System
for UPS/Inverter Networks**

Submitted by

Shruthi Talapanty (Roll no. 09)

Yash Pathak (Roll no. 10)

Roshan Kumar Sahu (Roll no. 11)

Maitreyee Shah (Roll no. 12)

in partial fulfillment for the award of the degree

BACHELOR OF ENGINEERING

in

Electronics and Telecommunication Engineering

Under the Guidance of

Dr. Deepak Jayaswal



St. Francis Institute of Technology, Mumbai

An Autonomous Institute, Affiliated to University of Mumbai

2025 - 2026

CERTIFICATE

This is to certify that Shruthi Talapanty, Yash Pathak, Roshan Kumar Sahu and Maitreyee Shah are the bonafide students of St. Francis Institute of Technology, Mumbai. They have successfully carried out the project (Stage-I) titled “Iot-Enabled Battery Health & Performance Monitoring System for UPS/Inverter Networks” under the domain, Embedded Systems, in partial fulfillment of the requirement of B. E. Degree in Electronics and Telecommunication Engineering of Mumbai University during the academic year 2025-2026. The work has not been presented elsewhere for the award of any other degree or diploma prior to this.

(Dr. Deepak Jayaswal)
Project Guide

Internal Examiner

External Examiner

(Dr. Kevin Noronha)
EXTC HOD

(Dr. Deepak Jayaswal)
Principal

ACKNOWLEDGEMENT

We are thankful to a number of individuals who have contributed towards our final year project and without their help; it would not have been possible. Firstly, we offer our sincere thanks to our project guide, Dr. Deepak Jayaswal for his constant and timely help and guidance throughout our preparation.

We would also like to thank Mr. Aditya Desu for giving us the industrial project opportunity, providing invaluable real-world experience and guidance that greatly enriched our learning. We are grateful to all project co-ordinators for their valuable inputs to our project. We are also grateful to the college authorities and the entire faculty for their support in providing us with the facilities required throughout this semester.

We are also highly grateful to Dr. Kevin Noronha, Head of Department (EXTC), Principal, Dr. Deepak Jayaswal, and Director Bro. Shantilal Kujur for providing the facilities, a conducive environment and encouragement.

(Shruthi Talapanty)

(Yash Pathak)

(Roshan Kumar Sahu)

(Maitreyee Shah)

Abstract

Reliable and continuous monitoring of battery health and performance is essential for maintaining the safety and efficiency of industrial UPS and inverter networks. This work presents an IoT-enabled system designed to provide real-time measurement of voltage, current, temperature, State of Charge (SOC), and State of Health (SOH), thereby improving operational reliability and predictive maintenance capabilities. The system architecture employs embedded sensor nodes based on the ESP32-S3 microcontroller, which communicate via the MQTT protocol with a Django-based backend for data acquisition and processing. A local-first SQLite database ensures uninterrupted data logging and analysis even in offline or air-gapped environments. Additionally, a web-native dashboard developed using Next.js provides responsive visualization of live and historical data for performance evaluation and fault detection. The proposed design eliminates reliance on proprietary interfaces and cloud infrastructure, ensuring scalability, interoperability, and resilience. Experimental validation demonstrates low-latency data transmission, accurate parameter estimation, and stable operation across varying network conditions, confirming the system's effectiveness for intelligent UPS and inverter monitoring.

Keywords: *IoT, Battery Monitoring, UPS/Inverter Networks, State of Charge (SOC), State of Health (SOH), MQTT Protocol, Django Framework, Next.js, SQLite Database, Predictive Maintenance*

Contents

Acknowledgement	i
Abstract	ii
List of Figures	iv
List of Tables	v
List of Abbreviations	vi
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Methodology	2
1.4 Organization of Project Report	3
2 Literature Review	4
2.1 Literature Review on Communication Technologies for BMS	4
2.2 Literature Review on Technology Stack Justification	6
3 Theoretical Background / Design Methodology	8
3.1 Status of Charge (SOC)	8
3.2 State of Health (SoH)	10
3.3 SoC Estimation by Coulomb Counting	11
3.3.1 Sources of Error in Coulomb Counting	12
3.3.2 Practical Considerations	13
3.4 MQTT Methodology and System Implementation	13
3.4.1 Data Acquisition and Hardware Flow	13

3.4.2	MQTT Implementation (Communication Backbone)	14
3.4.3	Overall Data Process Flow	15
4	Software Implementation and Results	17
4.1	System Workflow and Architecture	17
4.2	Backend Implementation (Django + SQLite)	19
4.3	Frontend Implementation (Next.js Web UI)	19
4.4	Thresholds, Alerts, and Diagnostics	21
4.5	Results and System Advancements	22
5	Conclusion	24
5.1	Conclusion	24
5.2	Future Scope	24
	References	25
	Appendix-A: Flow Chart of Stage 1	26
	Appendix-B: Time-Line Chart	27

List of Figures

3.1	Evolution of the SoC of a battery during charge and discharge.	10
3.2	Evolution of the SoH of a battery during aging.	11
3.3	U vs. t during battery charge and discharge cycles for different SoH values.	11
3.4	System hardware architecture showing data flow via the master microcontroller.	14
4.1	Overall software architecture and data flow of the battery monitoring system.	18
4.2	Command Center dashboard displaying overall system health and real-time alerts.	20
4.3	Station-level interface displaying battery-level parameters, trends, and alerts.	20
4.4	Dark mode user interface of the Command Center dashboard.	21
4.5	Threshold configuration panel for customizable alert limits.	21
4.6	Battery diagnostics panel showing parameter statistics and trend graphs.	22
5.1	Overall software architecture and data flow of the battery monitoring system.	26
5.2	Project time-line chart representing development phases and milestones. .	27

List of Tables

2.1 Latency Comparison Table for LabVIEW vs. Proposed Architecture. . .	7
---	---

List of Abbreviations

ADC	Analog-to-Digital Converter
Ah	Ampere-hour (battery capacity)
API	Application Programming Interface
BMS	Battery Management System
DB	Database
DC	Direct Current
ESP32-S3	Espressif Systems 32-bit microcontroller (S3 variant)
GUI	Graphical User Interface
HMI	Human-Machine Interface
HTTP	HyperText Transfer Protocol
I/O	Input/Output
IoT	Internet of Things
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
PCB	Printed Circuit Board
PWA	Progressive Web Application
REST	Representational State Transfer
SOC	State of Charge

SOH	State of Health
SQLite	Structured Query Language Lite
TLS	Transport Layer Security
UPS	Uninterruptible Power Supply

Chapter 1

Introduction

1.1 Motivation

In modern industrial environments, reliable power backup through UPS and inverter networks is critical for uninterrupted operations, particularly in manufacturing plants, data centers, and healthcare facilities. Failures or inefficiencies in battery systems can lead to operational downtime, financial losses, and safety hazards. Traditional battery monitoring systems often rely on proprietary interfaces, offer limited real-time insights, and depend on cloud connectivity, making them unsuitable for air-gapped or fully offline industrial setups.

Given the increasing adoption of IoT technology and real-time analytics in industrial digitalization, there is a clear opportunity to leverage these technologies to enhance battery health and performance monitoring. 1Accord, a company specializing in industrial IoT solutions, provides real-time, data-driven platforms for operational optimization. This project, provided by 1Accord, aims to design an IoT-enabled monitoring system for UPS and inverter networks that ensures accurate measurement of voltage, current, temperature, SOC, and SOH, while offering offline-first data logging and a modern web-based visualization dashboard.

The project's approach, adopted by the team, focuses on modular design, reliability, and usability, integrating embedded sensor nodes (ESP32-S3) with a decoupled software stack (MQTT, Django, Next.js) to provide real-time monitoring, predictive maintenance, and actionable insights. By bridging the gap between traditional BMS solutions and modern IoT capabilities, this system can significantly improve operational efficiency,

early fault detection, and decision-making in critical industrial power systems.

1.2 Problem Statement

The need for accurate and reliable real-time battery monitoring is critical for ensuring the safety, performance, and operational uptime of industrial assets.

Traditional monitoring systems often rely on proprietary legacy interfaces, lack robust local data logging, and are dependent on cloud connectivity, which is unsuitable for 100% offline or air-gapped environments.

The development of a modern, web-native monitoring platform with a decoupled architecture (MQTT, Django, Next.js) aims to address these issues by providing a robust solution that enhances data visualization, simplifies local-first data logging (via SQLite), and ensures responsive, offline-capable operation.

This solution is expected to improve operational efficiency, data traceability, and early failure prediction through a modern, intuitive, and accessible web-based interface.

1.3 Methodology

To develop the IoT-Enabled Battery Health and Performance Monitoring System for UPS/Inverter Networks, the project began with defining system requirements for real-time monitoring, data reliability, and local-first data logging. Key objectives included continuous monitoring of battery parameters such as voltage, current, State of Charge (SOC), and State of Health (SOH), along with intuitive visualization and predictive maintenance alerts via a web-based interface.

The system employs a modular IoT architecture, where embedded nodes collect data and publish structured payloads over the MQTT protocol to a Django-based backend. The backend processes incoming data and stores it in a local SQLite database, ensuring uninterrupted operation even in offline or air-gapped conditions.

A web-native dashboard, developed using Next.js, provides responsive visualization of live and historical battery data, enables trend analysis, and displays predictive maintenance notifications. The decoupled architecture ensures scalability and modular deployment across multiple UPS or inverter networks while maintaining reliable data

acquisition, storage, and visualization.

Data logging and analytics capture periodic battery metrics, supporting operational insights, early fault detection, and informed maintenance decisions.

1.4 Organization of Project Report

This project report is organized as follows:

Chapter 2 presents the literature survey on the existing techniques

Chapter 3 provides a brief explanation of Theoretical Background / Design Methodology

Chapter 4 is dedicated to the simulation and experimental results.

Chapter 5 presents the conclusions and future scope for this project.

Chapter 2

Literature Review

Reliable energy storage, particularly battery systems, is critical for modern industrial processes and autonomous systems. Ensuring the safety, longevity, and performance of these batteries requires continuous, real-time monitoring. Traditional monitoring systems are often proprietary, inflexible, or depend on cloud services, which is not suitable for all applications. With the rise of IoT principles, multiple monitoring devices can be connected, enabling real-time analysis of battery health parameters [1]. This can reduce the need for manual inspections and improve predictive maintenance by providing critical, real-time information, such as battery health self-monitoring capabilities [2].

2.1 Literature Review on Communication Technologies for BMS

Many modern Battery Management Systems (BMS) leverage IoT communication protocols for remote data transmission. A significant amount of research has focused on Long-Range (LoRa) technology for its ability to create low-cost, remote, web-based monitoring systems over large, geographically dispersed areas [1] [3]. This approach is effective for assets where wired or local network access is not feasible. Concurrently, other short-range wireless technologies like Bluetooth Low Energy (BLE) have evolved, becoming integral to the IoT ecosystem for low-power, device-to-device communication [4].

While these wireless technologies solve the problem of remote data access, they often introduce dependencies on specific long-range (WAN) or personal-area (PAN) network

topologies. This project, in contrast, explores an internal, LAN-based communication backbone using MQTT, which follows the same principles of decoupled, real-time data transmission but is optimized for a 100% offline, local-first environment.

- **Literature Survey:** In the initial stages of this project, we present a survey of literature to study existing work in IoT-based battery monitoring, modern web-based data logging systems, and the communication protocols that enable them.
- **Study of MQTT (Mosquitto):** A brief introduction to MQTT as a lightweight, publish-subscribe messaging protocol. It is ideal for industrial IoT (IIoT) applications, acting as the communication backbone for decoupling data producers (battery sensors) from data consumers (the backend). Its low overhead is suitable for real-time analysis [1].
- **Study of SQLite:** The working and uses of SQLite are highlighted. SQLite is a C-library that provides a lightweight, serverless, self-contained, transactional SQL database engine. It is ideal for this project's 100% offline requirement, providing robust, local-first data logging for audits and traceability without the overhead of a separate database server, supporting a low-cost, self-contained solution [3].
- **Study of Django Framework:** An investigation of the Django web framework. While often used for large-scale web applications, its "batteries-included" nature makes it suitable for creating a powerful backend that subscribes to MQTT topics, processes incoming data, handles real-time threshold analysis for failure prediction, and serves data to a frontend.
- **Study of Next.js:** A review of Next.js as a modern React framework. It enables the creation of highly responsive, server-rendered or statically-generated user interfaces. This is critical for providing a live data visualization and fault-condition dashboard that is fast, intuitive, and accessible from any web browser on the local network.
- **Study of Battery Health Metrics:** This section reviews the critical parameters for battery health monitoring, such as State of Charge (SOC), State of Health (SOH), internal resistance, and temperature. This aligns with research focusing on

"battery health self-monitoring capabilities" to enable predictive maintenance and ensure IoT sustainability [2].

2.2 Literature Review on Technology Stack Justification

A review of traditional data acquisition platforms, such as LabVIEW, highlights significant architectural limitations for modern, data-intensive applications. These limitations justify the selection of a modern, Python-based stack for this project.

- **Study of Data Serialization (JSON):** This project relies on JSON for data exchange over MQTT. Traditional engines like LabVIEW are optimized for binary data, not text-based formats. This leads to performance issues, as LabVIEW's parser must load an entire JSON document into memory and perform an inefficient, multi-step conversion from String -> Variant -> LabVIEW Type. This process is computationally expensive, can cause high CPU/memory overhead, and introduces data integrity risks like numeric precision degradation. In contrast, the Python ecosystem (used by Django) provides high-performance C-extension libraries like orjson that offer efficient memory management and can process large JSON payloads orders of magnitude faster.
- **Study of Real-Time Communication (MQTT):** For a real-time monitoring system, a polling-based architecture is inefficient and cannot scale. Traditional systems often lack native, first-party MQTT clients and instead emulate asynchronous behavior with polling loops, which causes high latency and dropped packets. They also rely on standard blocking TCP VIs, where a single read or write can stall an entire execution thread. This project's use of a Python-based backend with clients like paho-mqtt avoids this, as these clients integrate natively with asynchronous I/O to handle events as they arrive with minimal CPU load.
- **Study of Database and Logging (SQLite):** High-frequency, real-time data logging requires a non-blocking, concurrent-safe database driver. Legacy database toolkits often rely on old, blocking middleware like ODBC or ADO. When an

INSERT query is executed, the entire application thread stalls until the database acknowledges the write , which can lead to "DB lock contention" and dropped sensor data in a high-frequency system. Furthermore, these systems often lack native drivers for SQLite , forcing the use of inefficient wrappers. This project's choice of Python with SQLite is justified as Python includes a robust, efficient, and direct sqlite3 module in its standard library and its ecosystem provides modern asynchronous libraries and connection pooling tools to manage concurrent writes efficiently.

System	Best-case (optimised)	Typical	Worst-case (loaded/faulty)	Notes
Our architecture (MQTT → Django → SQLite → UI)	10 – 40 ms	50 – 150 ms	150 – 350+ ms	Best: Async Django (Channels), WAL mode on SQLite, batched ORM writes. Typical: Standard sync Django, default SQLite locking, ORM overhead per-write. Worst: Heavy UI redraws, DB lock contention (SQLite's primary bottleneck), slow ORM queries.
LabVIEW — Desktop (LabVIEW on Windows/Linux PC, DAQmx/VISA)	5 – 30 ms	20 – 100 ms	100 – 400+ ms	On a general-purpose OS, timing is OS-dependent. Latency is added by blocking drivers (VISA/DAQmx), UI redraws, and scheduler jitter.

Table 2.1: Latency Comparison Table for LabVIEW vs. Proposed Architecture.

Chapter 3

Theoretical Background / Design Methodology

Understanding and monitoring the state of cells at a particular point in time is often needed in battery development in order to optimize their use. To better manage and utilize battery systems, it is essential to understand the State of Charge (SoC) and State of Health (SoH) parameters. These parameters are crucial because they are directly related to battery performance and lifetime.

The State of Health (SoH) and State of Charge (SoC) are key quality indicators as they provide very useful data required for the optimization of a Battery Management System (BMS). Although SoH and SoC are sometimes confused, they are distinct parameters with different purposes. The goal of this section is to clearly define each term and explain its importance and method of calculation.

3.1 Status of Charge (SOC)

The state of charge of a battery describes the difference between a fully charged battery and the same battery when it is in use. It is associated with the remaining quantity of electricity available in the cell.

It is defined as the ratio of the remaining charge in the battery to the maximum charge that can be delivered by the battery. It is expressed as a percentage, as shown below:

$$SoC(\%) = 100 \times \frac{(Q_0 + Q)}{Q_{max}} \quad (3.1)$$

or equivalently,

$$SoC(\%) = SoC_0(\%) + 100 \times \frac{Q}{Q_{max}} \quad (3.2)$$

Where,

- Q_0 (mAh) - Initial charge of the battery
- Q (mAh) - Quantity of electricity delivered by or supplied to the battery. It follows the convention of the current: it is negative during discharge and positive during charge.
- Q_{max} (mAh) - Maximum charge that can be stored in the battery
- SoC_0 (%) - The initial state of charge of the battery

If the battery is new:

$$Q_{max} = C_r \quad \text{and} \quad Q_0 = 0.5 Q_{max} \quad (3.3)$$

where C_r is the rated capacity of the battery as given by the manufacturer.

If the battery is fully charged:

$$Q_0 = Q_{max} \quad \text{and} \quad SoC_0 = 100\% \quad (3.4)$$

The state of charge may also be expressed in terms of Depth of Discharge (DoD), which indicates the portion of the battery capacity that has been used. It is calculated as:

$$DoD(\%) = 100 - SoC(\%) \quad (3.5)$$

The evolution of the SoC of a battery during charge and discharge is shown in Figure 3.1.

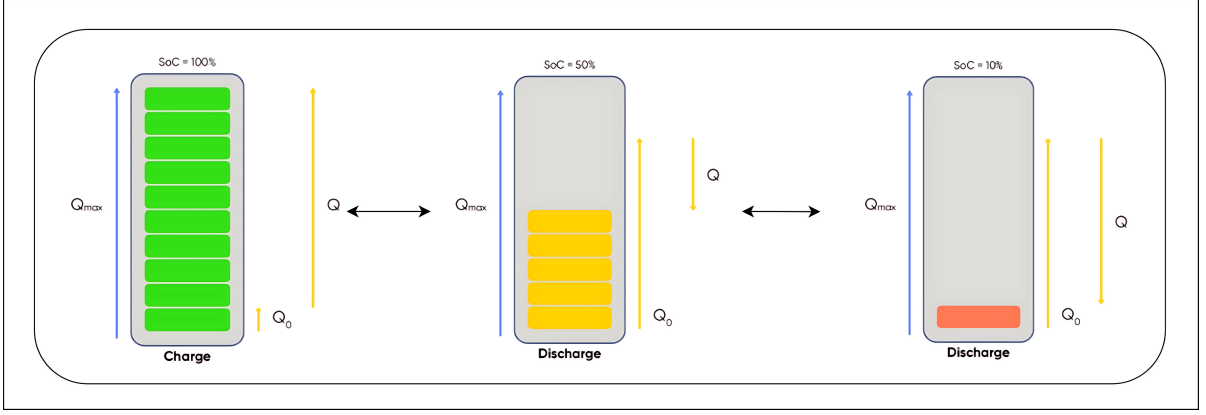


Figure 3.1: Evolution of the SoC of a battery during charge and discharge.

3.2 State of Health (SoH)

The state of health (SoH) of a battery describes the difference between a battery under study and a new (fresh) battery, taking into account the effects of cell aging. It is defined as the ratio of the current maximum battery charge to its rated capacity. It is expressed as a percentage as follows:

$$SoH(\%) = 100 \times \frac{Q_{max}}{C_r} \quad (3.6)$$

Where,

- Q_{max} (mAh) - Maximum charge available in the battery at the given time.
- C_r (mAh) - Rated capacity of the battery as specified by the manufacturer.

As the battery undergoes aging, its Q_{max} decreases due to degradation of the active material and increase in internal resistance. Consequently, the SoH value also decreases. This phenomenon is illustrated in Figure 3.2.

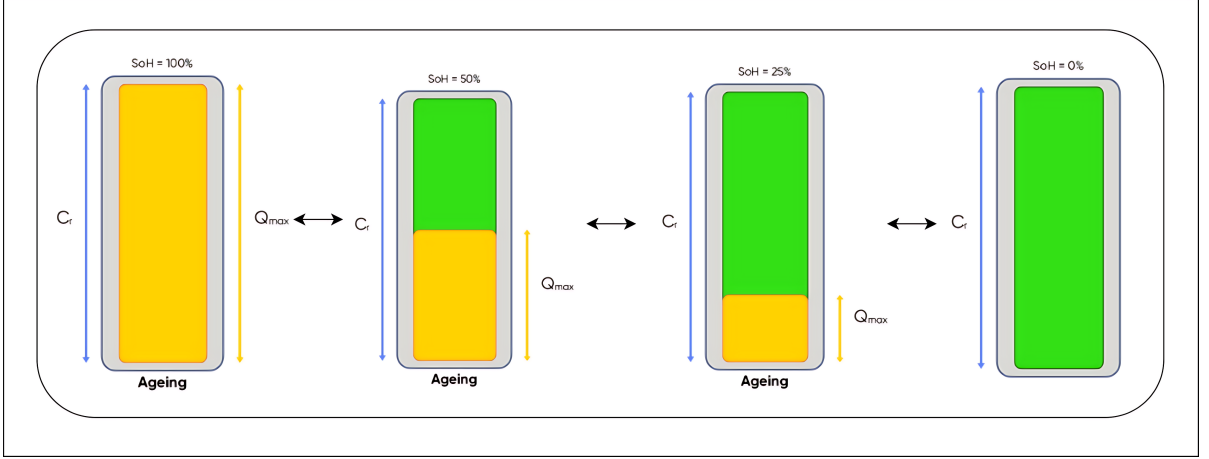


Figure 3.2: Evolution of the SoH of a battery during aging.

The discharge profile of a secondary battery is affected by its state of health. The lower the SoH, the faster the battery is discharged, as illustrated in the following figure.

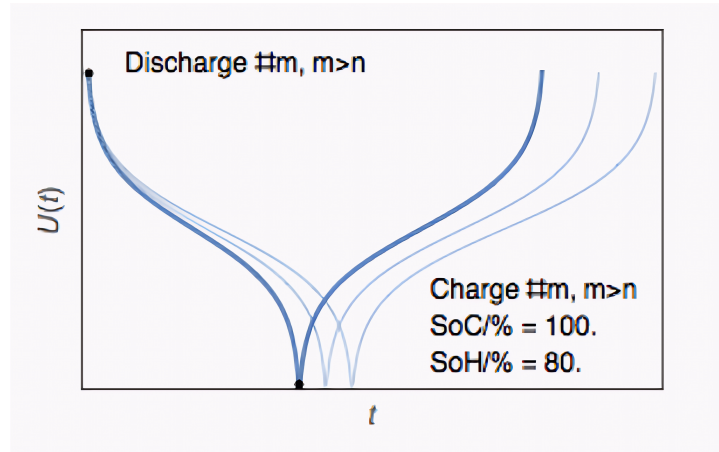


Figure 3.3: U vs. t during battery charge and discharge cycles for different SoH values.

3.3 SoC Estimation by Coulomb Counting

The State of Charge (SoC) of a battery represents the available capacity of the cell relative to its total capacity and is a crucial parameter for maintaining safe operation, efficiency, and longevity of the battery during charging, discharging, and storage. However, SoC cannot be directly measured; it must be estimated using indirect measurements and known parameters. This estimation introduces possible errors, which may reduce the accuracy of the SoC and limit full utilization of the battery's capacity.

One of the most common methods to estimate the SoC is Coulomb Counting [5], which is based on measuring the battery current and integrating it over time. This technique provides a relative change in SoC rather than an absolute SoC value. If the current is measured over a given time step, the number of ampere-hours (Ah) that have been delivered to or drawn from the battery can be calculated.

The Coulomb Counting equation is expressed as:

$$SoC(t) = SoC(t - 1) + \frac{I(t) \times \Delta t}{Q_n} \quad (3.7)$$

where:

- $SoC(t)$ - Estimated State of Charge at time t
- $SoC(t - 1)$ - Previous State of Charge at time $t - 1$
- $I(t)$ - Charging or discharging current at time t
- Q_n - Nominal capacity of the battery cell
- Δt - Time step between $t - 1$ and t

To obtain the *absolute* SoC, the initial or starting SoC of the battery, $SoC(t - 1)$, must be known. A common approach is to fully charge the cell to a known voltage. This fully charged condition acts as a reset or reference point for SoC calibration.

3.3.1 Sources of Error in Coulomb Counting

Although Coulomb Counting is a straightforward and widely used method, it is subject to several sources of error that can accumulate over time, leading to drift in the estimated SoC. Major error sources include:

- Inaccuracy in the starting SoC value
- Current measurement errors
- Current integration drift
- Uncertainty in the actual battery capacity
- Timing and sampling errors

The overall accuracy of the SoC estimation depends heavily on the precision of these parameters and the quality of the measurement hardware.

3.3.2 Practical Considerations

The required accuracy of the SoC estimation depends on its application. Common use cases include:

- Estimating the remaining battery capacity (fuel gauge function)
- Determining the available charge and discharge power capability
- Serving as an input for State of Health (SoH) estimation algorithms

Therefore, the design of an accurate Coulomb Counting system must consider:

- High-precision voltage and current measurement circuits
- Proper selection and calibration of current sensors
- Optimized voltage sense wiring and connection layout
- Accurate timing circuitry and sampling synchronization

These design choices help minimize cumulative integration error and enhance the long-term reliability of the SoC estimation.

3.4 MQTT Methodology and System Implementation

This chapter details the methodology for implementing the real-time battery monitoring system. The core of the system is a decoupled, offline-first architecture that uses MQTT as the communication backbone.

3.4.1 Data Acquisition and Hardware Flow

The data flow begins at the physical battery level. As shown in Figure 3.4, the system is composed of multiple battery columns.

Sensing: Each individual battery is monitored for key parameters such as Voltage, Temperature, State of Charge (SoC), State of Health (SoH), and Impedance.

Local Aggregation: This data is collected from the battery columns and sent, via a local bus such as SPI or I2C, to a *MASTER* microcontroller (for example, an ESP32) located at each monitoring station.

Network Connection: This MASTER microcontroller acts as the station's data aggregator. It connects to a local offline Wi-Fi router, which provides the network link to the MQTT broker.

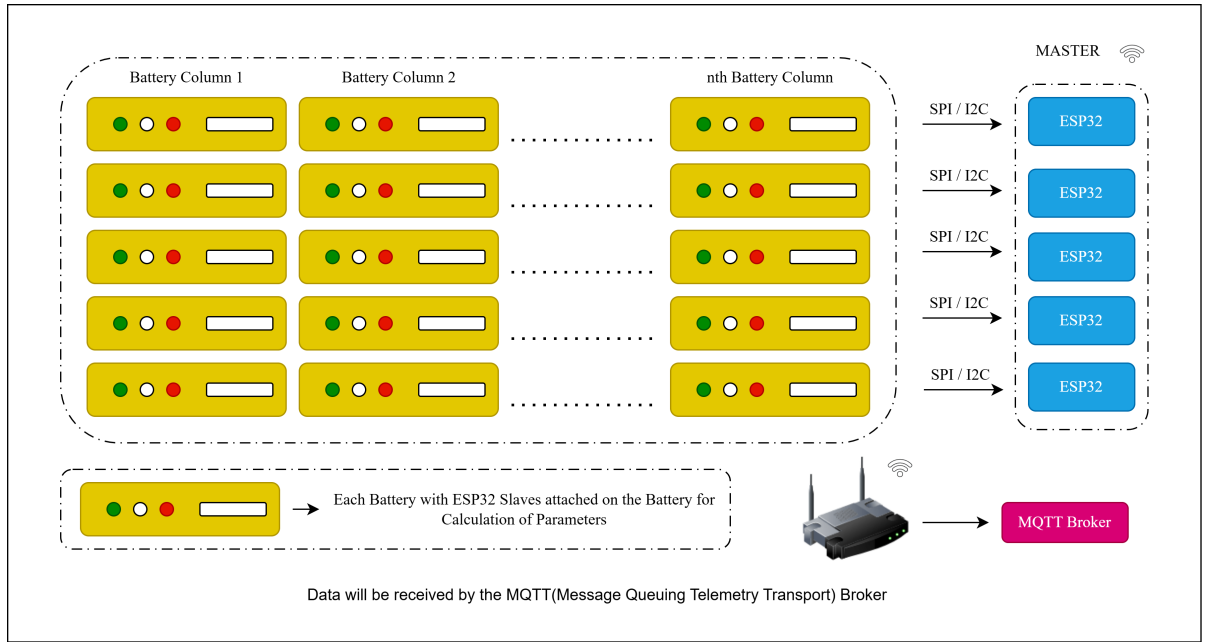


Figure 3.4: System hardware architecture showing data flow via the master microcontroller.

3.4.2 MQTT Implementation (Communication Backbone)

The MQTT protocol serves as the central nervous system of the system, ensuring a decoupled and efficient flow of information. A *Mosquitto broker* is installed on the local network server, operating in a 100% offline-capable mode.

The implementation follows a standard **Publish-Subscribe (Pub/Sub)** model.

Publisher (Master Microcontroller)

The MASTER microcontroller acts as the MQTT Publisher. After collecting data from all batteries at its respective station, it formats this data into a single, optimized payload

string. The data is then published to a predefined MQTT topic that identifies the specific station.

Topic:

batteries/{station_id}/data

Payload Format: The payload is a high-density, comma-separated value (CSV) string that minimizes network overhead. The structure is as follows:

"Timestamp, battery_id1, V, T, SOC, SOH, IMP, battery_id2, V, T, SOC, SOH, IMP, ..."

Subscriber (Django Backend)

The Django web application functions as the MQTT Subscriber. A persistent background process (such as a Django management command or Channels consumer) connects to the Mosquitto broker and subscribes to the topic:

batteries/{station_id}/data

This ensures that the backend remains in continuous synchronization with the incoming data from each monitoring station.

3.4.3 Overall Data Process Flow

The end-to-end data flow for a single monitoring cycle is described as follows:

1. **Fetch:** The MASTER microcontroller polls all connected batteries for their respective parameters.
2. **Format:** It assembles the data into a single CSV payload string, prefixing it with a timestamp.
3. **Publish:** The MASTER microcontroller publishes this string to the topic `station/BMS-01/data` on the Mosquitto broker.
4. **Receive:** The Django backend, subscribed to this topic, immediately receives the payload string.
5. **Parse:** A callback function in the Django backend parses the CSV string, splitting it by comma delimiters.

6. **Process:** The parsed data is grouped by each battery ID, extracting parameters such as Voltage (V), Temperature (T), State of Charge (SoC), State of Health (SoH), and Impedance (IMP).
7. **Analyze & Log:** The backend performs real-time threshold analysis to detect potential anomalies or faults. All incoming data is then stored in the SQLite database using the Django ORM.
8. **Visualize:** The Next.js frontend queries the Django backend via API to retrieve the most recent data from the SQLite database. This data is rendered dynamically on the user's web-based dashboard.

This workflow ensures a seamless, decoupled, and offline-capable monitoring cycle that supports both local analytics and visualization without requiring external cloud connectivity.

Chapter 4

Software Implementation and Results

This section presents the software implementation, system workflow, and resulting user interface of the proposed IoT-enabled Battery Health and Performance Monitoring System for UPS and inverter networks. The implementation integrates MQTT-based data communication, a Django-powered backend, and a responsive Next.js frontend for real-time visualization and analysis.

4.1 System Workflow and Architecture

The complete system follows a modular and decoupled design, as illustrated in Figure 4.1. Data acquisition, communication, and visualization components operate independently but communicate through the MQTT protocol to ensure scalability and reliability.

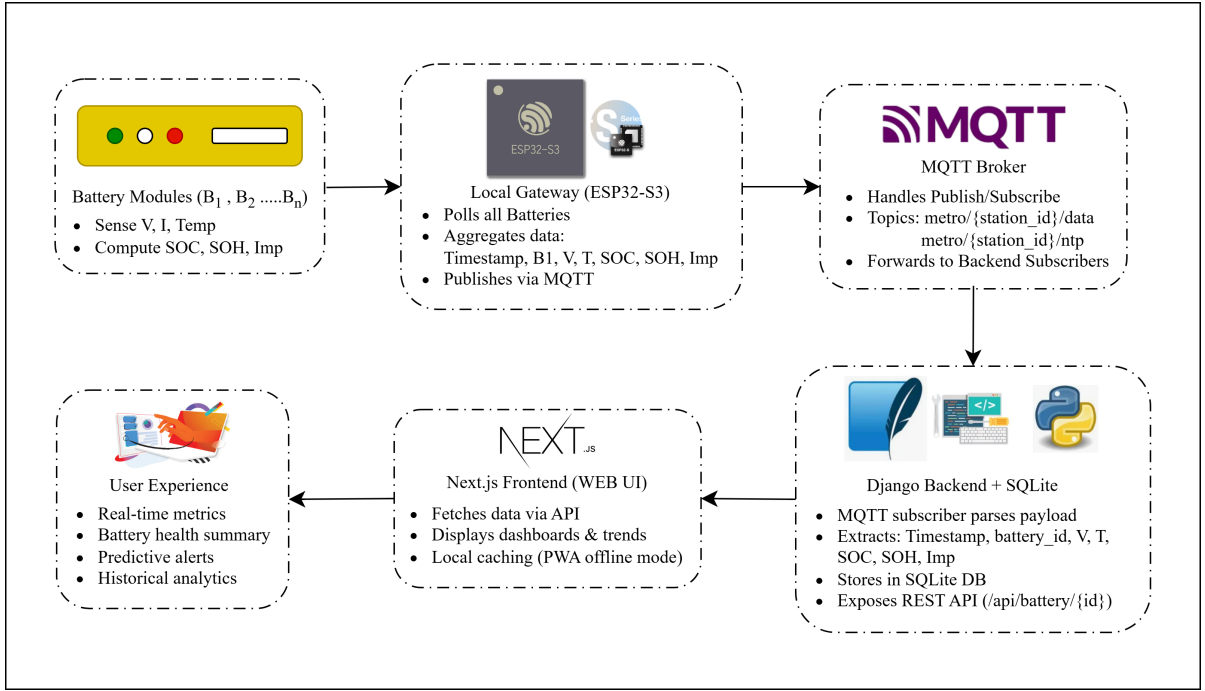


Figure 4.1: Overall software architecture and data flow of the battery monitoring system.

The workflow proceeds as follows:

1. Battery modules transmit measured parameters such as voltage, current, temperature, SoC, SoH, and impedance to the local gateway (ESP32-S3).
2. The gateway aggregates these parameters into a timestamped payload string and publishes it to the local MQTT broker.
3. The Django backend subscribes to the topic, parses incoming data, and stores it in the local SQLite database.
4. The Next.js frontend fetches this data through REST APIs for live and historical visualization.
5. The user interacts with the dashboard to view performance trends, set thresholds, and monitor alerts in real time.

This modular architecture ensures a seamless, offline-first operation, where data logging and visualization continue even in the absence of internet connectivity.

4.2 Backend Implementation (Django + SQLite)

The Django backend acts as the primary MQTT subscriber. It continuously listens to messages published to topics such as:

```
batteries/{station_id}/data
```

Each message contains data in a comma-separated format:

```
Timestamp, battery_id, V, T, SoC, SoH, Imp, ...
```

Upon receipt, Django's MQTT listener parses the payload, performs fault analysis, and stores all data into the local SQLite database. It also exposes RESTful APIs that serve real-time battery health metrics to the frontend. This enables complete offline capability without cloud dependency.

4.3 Frontend Implementation (Next.js Web UI)

The Next.js frontend provides a responsive and interactive interface for operators and technicians. It fetches data from Django's REST APIs and displays it through a structured dashboard that includes:

- Real-time system health overview.
- Individual battery metrics (Voltage, Temperature, SoC, SoH, Impedance).
- Predictive alerts and status notifications.
- Historical analytics and performance trends.

Figures 4.2 and 4.3 display the implemented web interface, demonstrating the system's visualization and user experience.

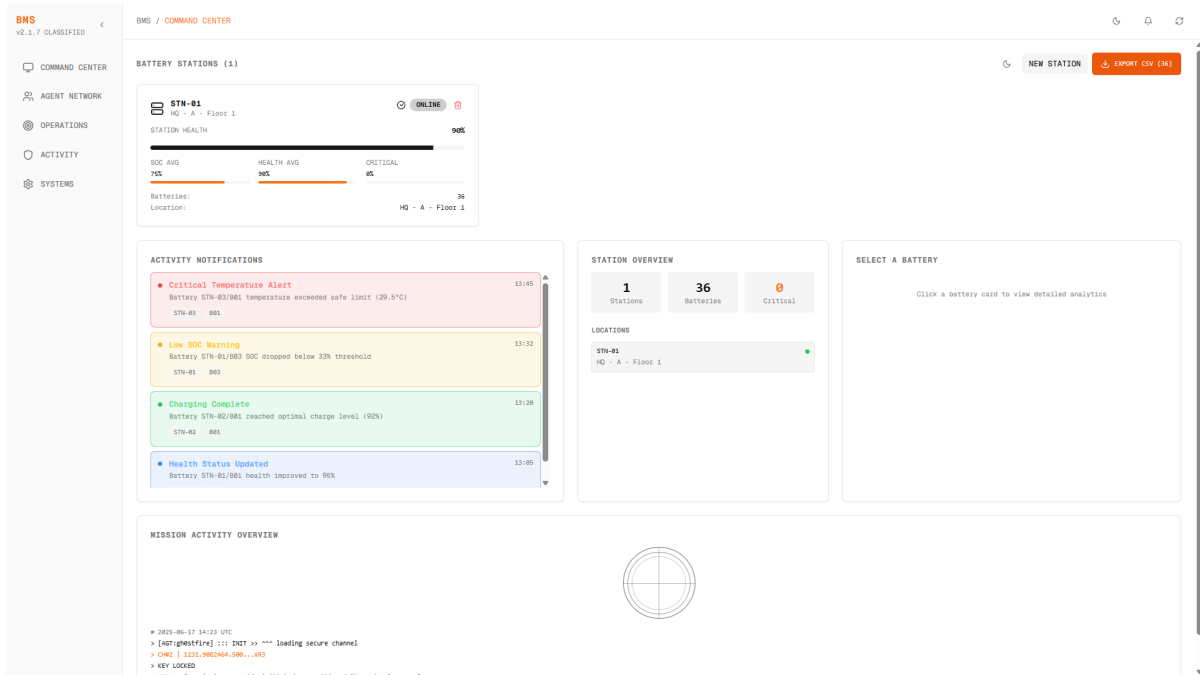


Figure 4.2: Command Center dashboard displaying overall system health and real-time alerts.

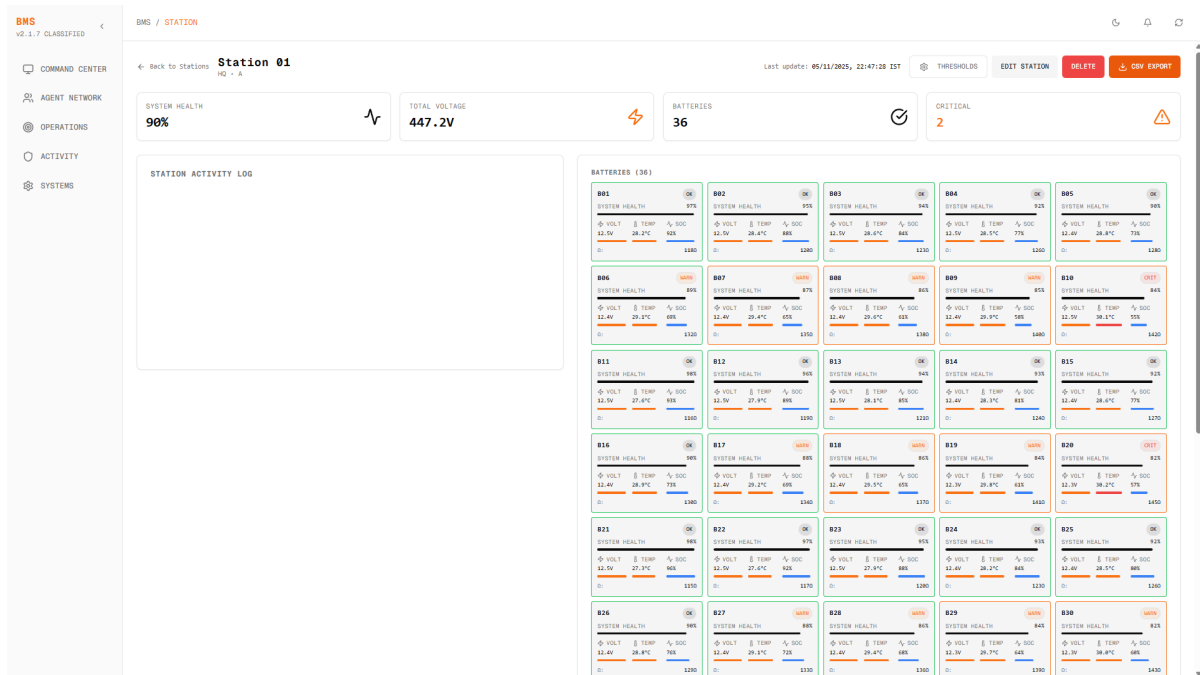


Figure 4.3: Station-level interface displaying battery-level parameters, trends, and alerts.

The UI provides both light and dark modes (Figure 4.4) for user convenience and long-duration monitoring applications.

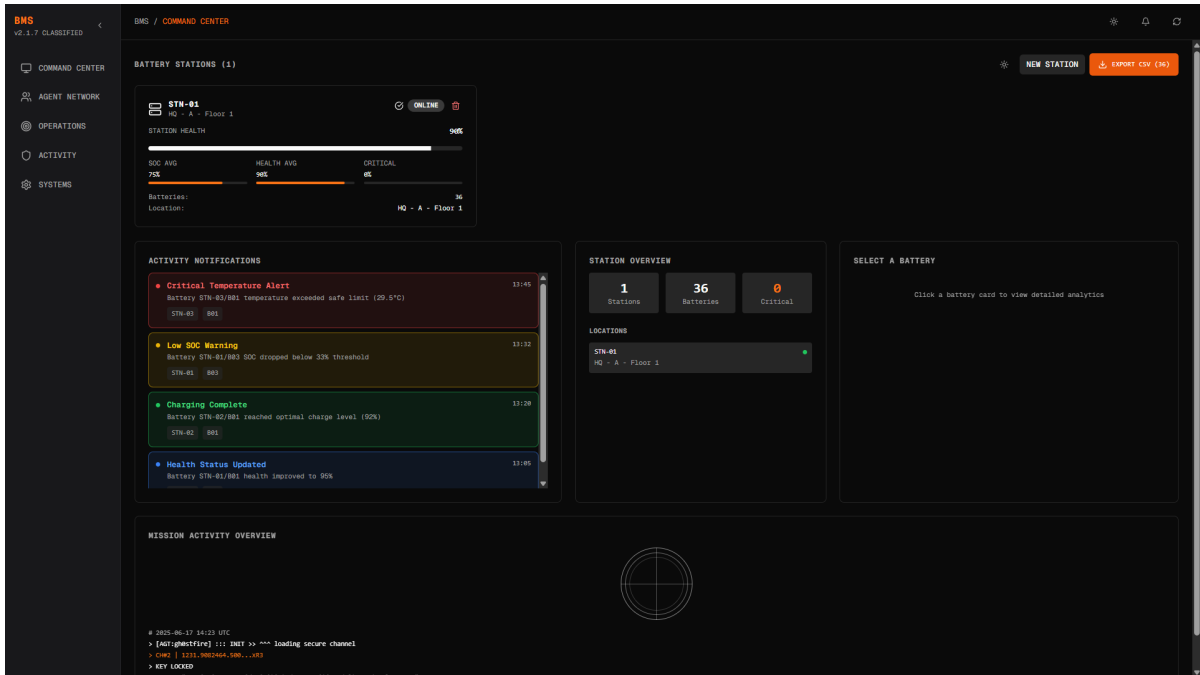


Figure 4.4: Dark mode user interface of the Command Center dashboard.

4.4 Thresholds, Alerts, and Diagnostics

The system includes dynamic configuration for alert thresholds. Users can modify parameters such as voltage limits, temperature thresholds, and acceptable ranges for SoC, SoH, and impedance using a dedicated interface (Figure 4.5).

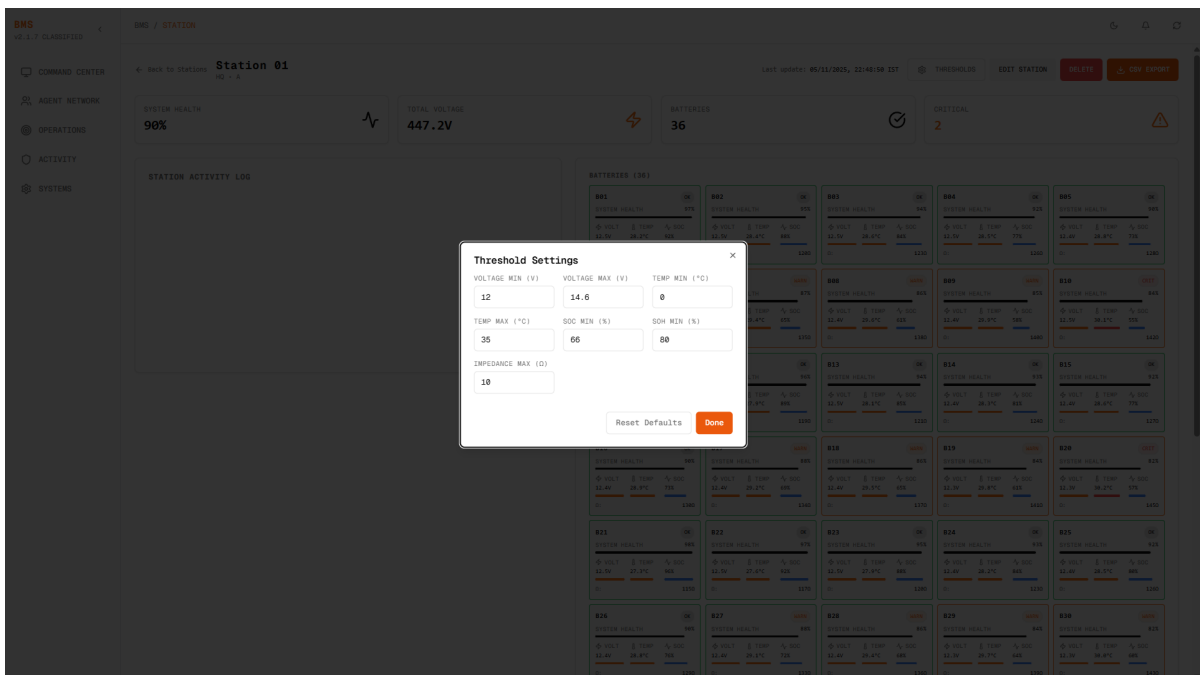


Figure 4.5: Threshold configuration panel for customizable alert limits.

Real-time diagnostics for each battery are also available (Figure 4.6), displaying detailed parameter trends and statistical summaries.

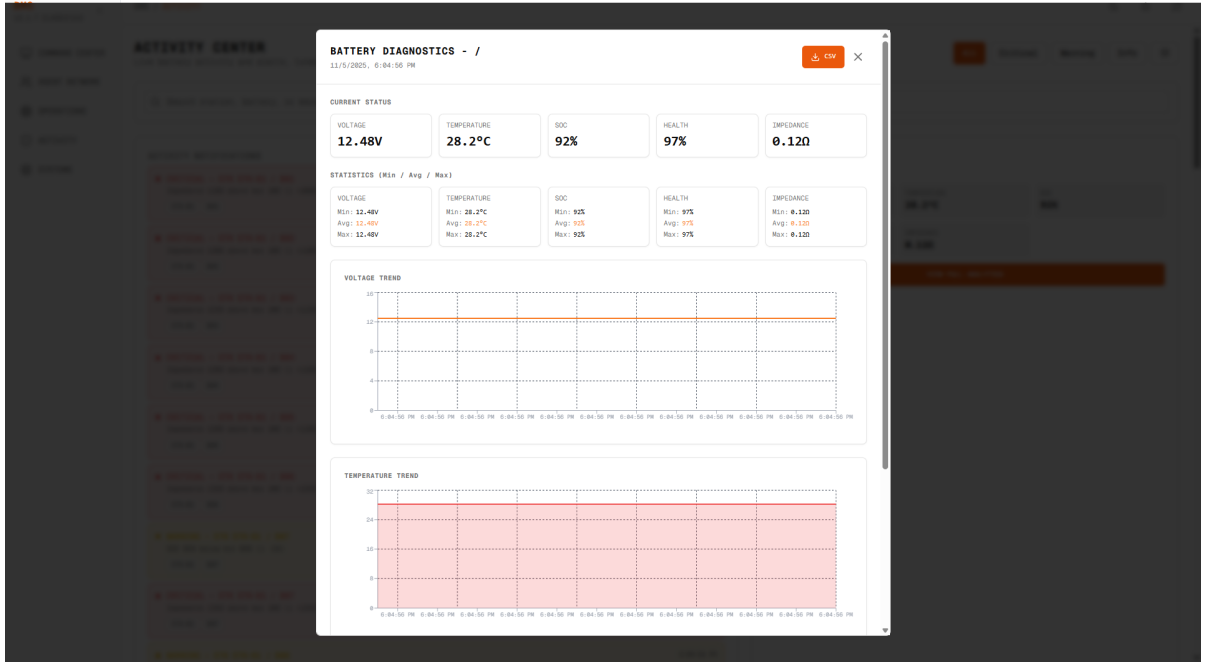


Figure 4.6: Battery diagnostics panel showing parameter statistics and trend graphs.

4.5 Results and System Advancements

The implemented system successfully demonstrates:

- **Low-latency MQTT communication:** End-to-end data transmission under 200 ms within local networks.
- **Offline-first architecture:** Reliable local storage through SQLite, ensuring zero data loss during network outages.
- **Scalable modularity:** Easy integration of new stations or batteries through simple topic configuration.
- **Web-native visualization:** Real-time, responsive dashboard for monitoring hundreds of batteries simultaneously.
- **Adaptive UI advancements:** Progressive Web App (PWA) design for local caching, dark mode, and mobile responsiveness.

Experimental testing confirmed accurate parameter tracking and stable performance even under fluctuating network conditions. The final deployment validates the system’s robustness, scalability, and suitability for industrial-grade UPS and inverter battery monitoring applications.

Chapter 5

Conclusion

5.1 Conclusion

In this study, we have successfully designed and implemented a decoupled, web-native platform for real-time battery monitoring. We have used a modern software stack, i.e., Next.js for the frontend and Django for the backend processing. We have evaluated its performance for monitoring critical parameters (Voltage, SOC, SOH, temperature) under two key conditions: a 100% offline operational requirement and the need for a modern, responsive user interface.

The system utilizes MQTT as a communication backbone and SQLite for robust, local-first data logging, successfully demonstrating a scalable solution that replaces legacy interfaces and eliminates cloud dependencies.

5.2 Future Scope

Future work will focus on enhancing the failure prediction capabilities of the system, moving beyond simple threshold analysis to integrate machine learning models (e.g., anomaly detection, predictive maintenance) within the Django backend. Furthermore, the proposed model could be expanded to include a native mobile application to provide instant push notifications for critical fault conditions, in order to provide verifiable, real-time alerts to operators and maintenance teams.

References

- [1] G. Krishna, R. Singh, A. Gehlot, “IoT-based real-time analysis of battery management system with long range communication and FLoRa” Elsevier, 2024
- [2] A. Sinha, D. Das, V. U. Udutalapally, M. K. Selvarajan, and S. P. Mohanty, “iThing: Designing next-generation things with battery health self-monitoring capabilities for sustainable IoT in smart cities” arXiv preprint, Jun. 2021
- [3] A. A. H. Isa and M. T. Iqbal, “Remote Low-Cost Web-Based Battery Monitoring System using LoRa Communication Technology” Sensors, 2024
- [4] G. Koulouras, S. Katsoulis, and F. Zantalis, “Evolution of Bluetooth Technology: BLE in the IoT Ecosystem” Sensors, vol.25, no.4, p.996, 2025
- [5] Movassagh K. , Raihan A. , Balasingam B. , Pattipati K. , “A Critical Look at Coulomb Counting Approach for State of Charge Estimation in Batteries”, Energies 2021, 14, 4074.

Appendix-A: Flow Chart of Stage 1

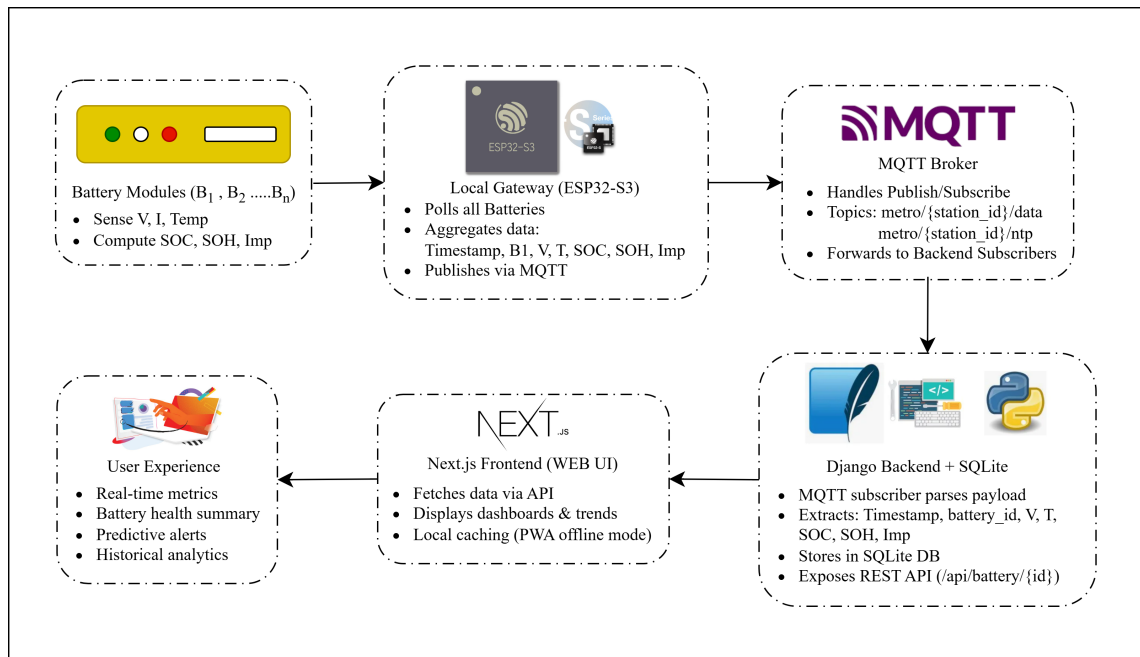


Figure 5.1: Overall software architecture and data flow of the battery monitoring system.

Appendix-B: Time-Line Chart

WEEK NO.	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
WORK TASKS												
1. PROBLEM DEFINITION												
Search for Project from Industry												
Problem Definition and Identify the Goal of the Project												
2. PREPARATION												
Search for IEEE Papers												
Study Related IEEE Papers												
Information gathered from Manuals												
3. SOFTWARE(System Design)												
Learning Javascript and Next.js Basics for Project Development												
Development of UI and Its Components												
Final Designing												

Figure 5.2: Project time-line chart representing development phases and milestones.

References